

Breaking AES with an Oscilloscope

Nathan Jones

Breaking AES with an Oscilloscope

Introduction

Like a screen door on a submarine

What's "differential power analysis" (DPA)?

A brief overview of AES encryption

Conducting a DPA attack

Setting up the attack

Conducting the attack

Defending against a DPA attack

Conclusion

Resources and going further

Introduction

Network security is a well-studied (albeit constantly evolving) field and the designers of internet-connected electronic devices all (hopefully) know at this point that they need to take network security seriously. But equally important are the threats to a device when an attacker has physical access to it! Think about it: how confident are you that your computer is safe from remote attacks when it's on your home network? (Probably fairly high.) Compare that to how confident you would be if you physically left your computer in the hands of an attacker for a week.



She's not getting that back.

The latter is far scarier because in that scenario the attacker can probe signals, modify PCBs, and more, to compromise the device! The field of **hardware security** studies these types of attacks and their countermeasures. I'd warrant that they are just as important as network security, though far less understood. In this article, I'm going to seek to remedy that by showing you (one example of) what hardware threat actors are capable of. The more you know, the better you can thwart a would-be hardware attacker in the future!

We're going to attack a fictitious device that's running AES encryption (specifically, AES128 in ECB mode). We'll assume that we can send the device some plaintext to be encoded, but that we don't see the ciphertext it generates. Then, with just an oscilloscope and some math, we'll be able to extract the private AES key inside that device in only a few minutes!

Like a screen door on a submarine

In policing, good detective work relies on the fact that no action comes side-effect free. A picked lock shows scratch marks. An object picked up and placed back down in the exact same location may have fingerprints left on its sides. Dust or dirt where there shouldn't be (or absent where it is supposed to be) can indicate a person's presence or absence.

Purrlock Holmes here to
investigate why the food bowl is
half empty



Consider that fictitious electronic device, the one that's doing the AES encryption. In much the same way that every physical action comes with side effects, the actions of that electronic device come with side effects as well. It's not *just* encrypting a message, it's also drawing power and generating heat. It's executing the encryption over a very specific length of time. It's generating error messages to specific incorrect inputs. Each of these qualities (power consumption, heat, execution time, and faulty outputs, respectively) represents a way in which an electronic device "leaks" information about what it's doing and, just like fingerprints and dust, all

of these things tell us a little bit about what's going on inside that electronic device. This leaked information is called a "side-channel" and using it to infer what a device is doing is called a "side-channel attack".

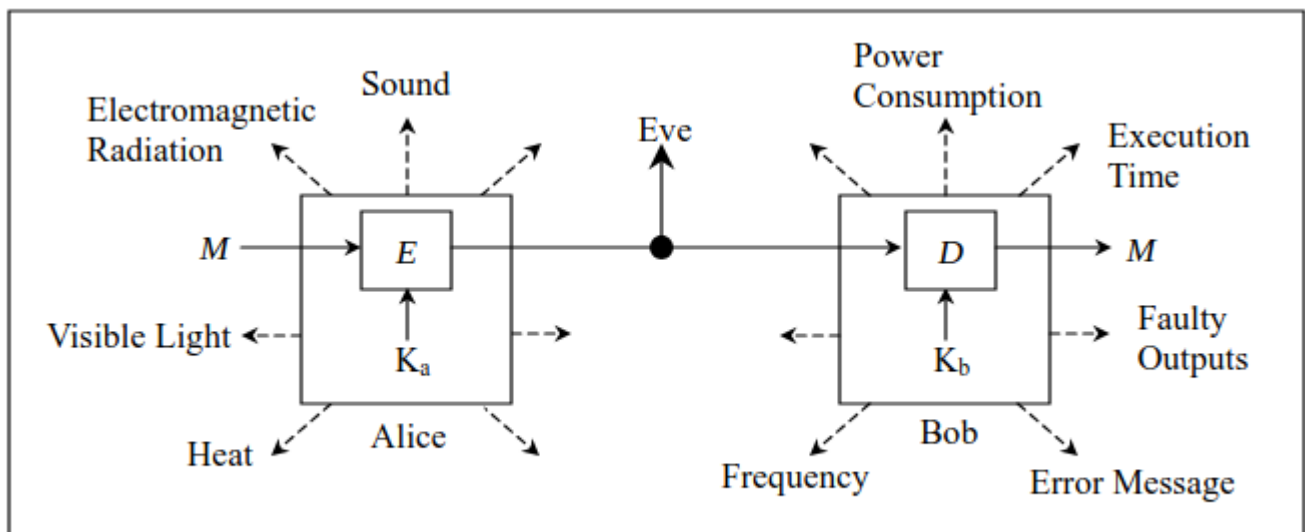


Figure 2: The cryptographic model including side-channel

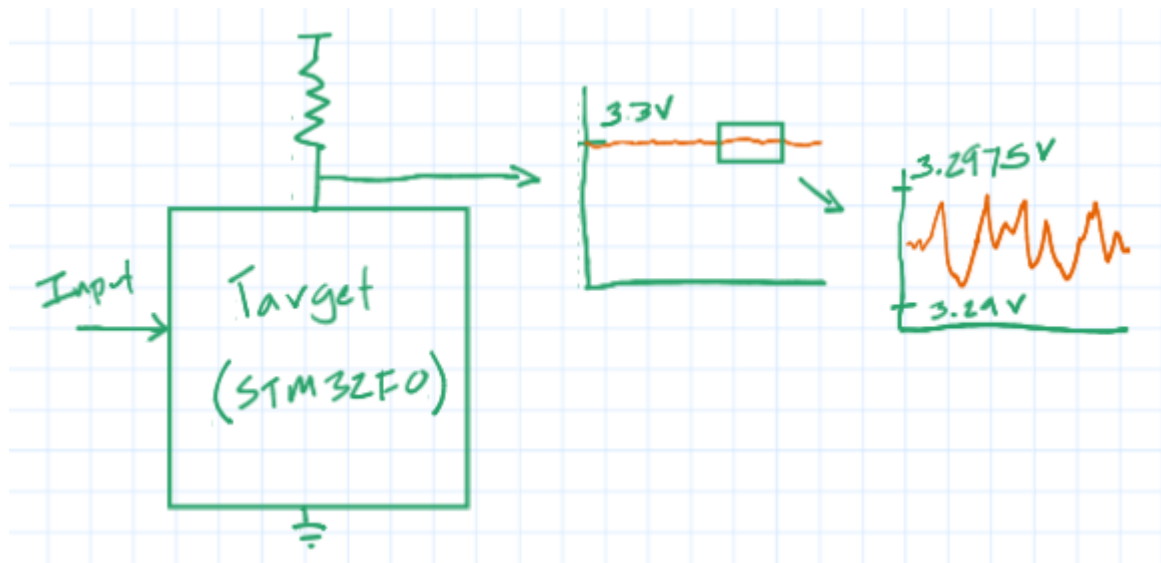
Zhou, Yongbin & Feng, DengGuo. (2005). Side-Channel Attacks: Ten Years After Its Publication and the Impacts on Cryptographic Module Security Testing.. IACR Cryptology ePrint Archive. 2005. 388.
<https://eprint.iacr.org/2005/388.pdf>

Have you ever forgotten a website login, been told "You've entered an incorrect password", and thought to yourself, "Aha! My username, at least, must be correct." If so, you've successfully conducted a small side-channel attack on that website, by leveraging the fact that the faulty output ("You've entered an incorrect password") leaked information that your username was (likely) correct, otherwise the website would have flagged the username as being incorrect first.

With the right tools (and more than a little math), however, we can do more than just *infer* small bits of information about what an electronic device is doing. In some cases, we can actually **recover the private key on a device performing AES encryption simply by watching how much power it draws with an oscilloscope**. The attack that we'll conduct to achieve that is called a **differential power analysis** attack. And that, my friends, is about as secure as a screen door on a submarine.

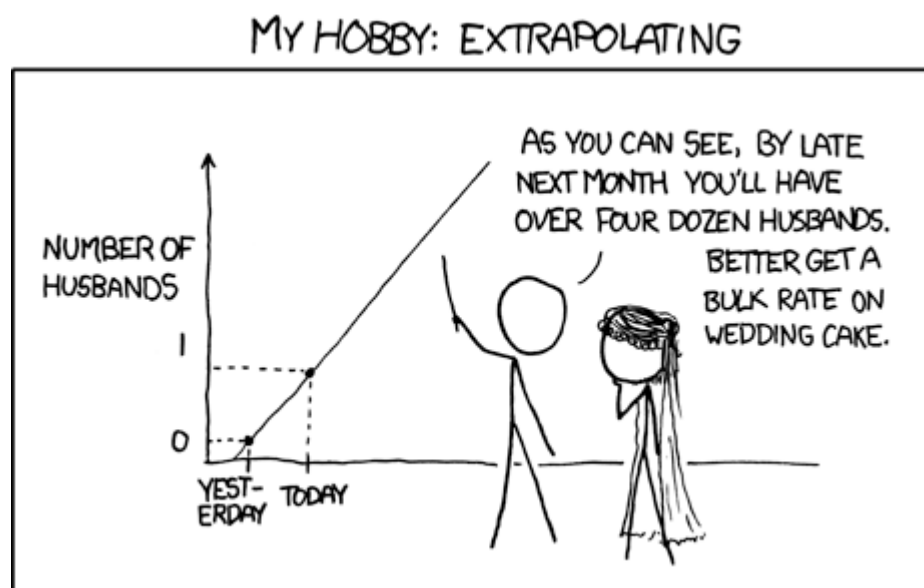
What's "differential power analysis" (DPA)?

The "power analysis" in "differential power analysis" comes from the fact that we're choosing an electronic device's power consumption as our side-channel. For a basic setup, all we need is a shunt resistor in line with our target device's V_{cc} pin and an oscilloscope connected to the bottom node.



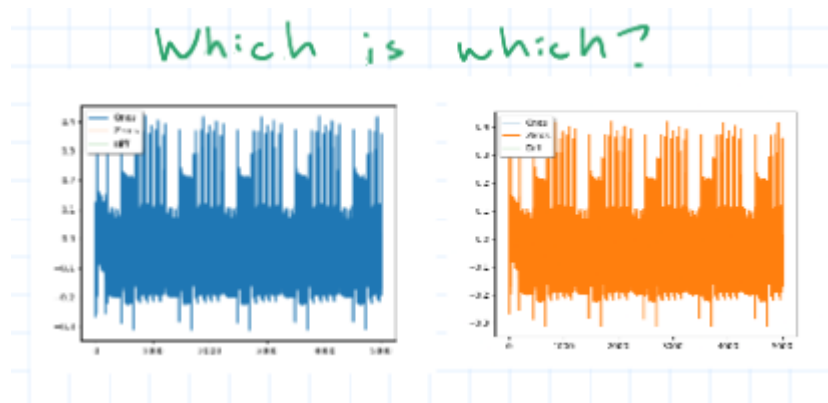
Filtering capacitors can hide the minuscule power fluctuations we're trying to measure, so as many of those as can be removed before the target becomes unstable should be removed. In the diagram above, we're taking a single-ended measurement of the voltage above the microcontroller and assuming that the power supply voltage remains constant, but a slightly more advanced setup would take a differential measurement across the shunt resistor. (NOTE: This is *not* where the "differential" in DPA comes from.)

Now, it should not come as a shock that we can use this setup to infer broad things like "the microcontroller is doing a lot of work right now" or "it's probably in a sleep mode"; higher CPU activity and more things that are "on" in a microcontroller *do* correlate with higher power consumption, exactly because it takes energy to turn on a transistor. In fact, the amount of energy a microcontroller uses at a given moment in time is *directly* and *linearly* correlated with how many transistors in that microcontroller are "on" and this is key for allowing the DPA attack to work. As you can imagine, however, the contribution of one transistor in the overall power trace of a microcontroller is vanishingly small; almost assuredly too small for us to measure even with a \$10,000 oscilloscope. And so, we turn to statistics!

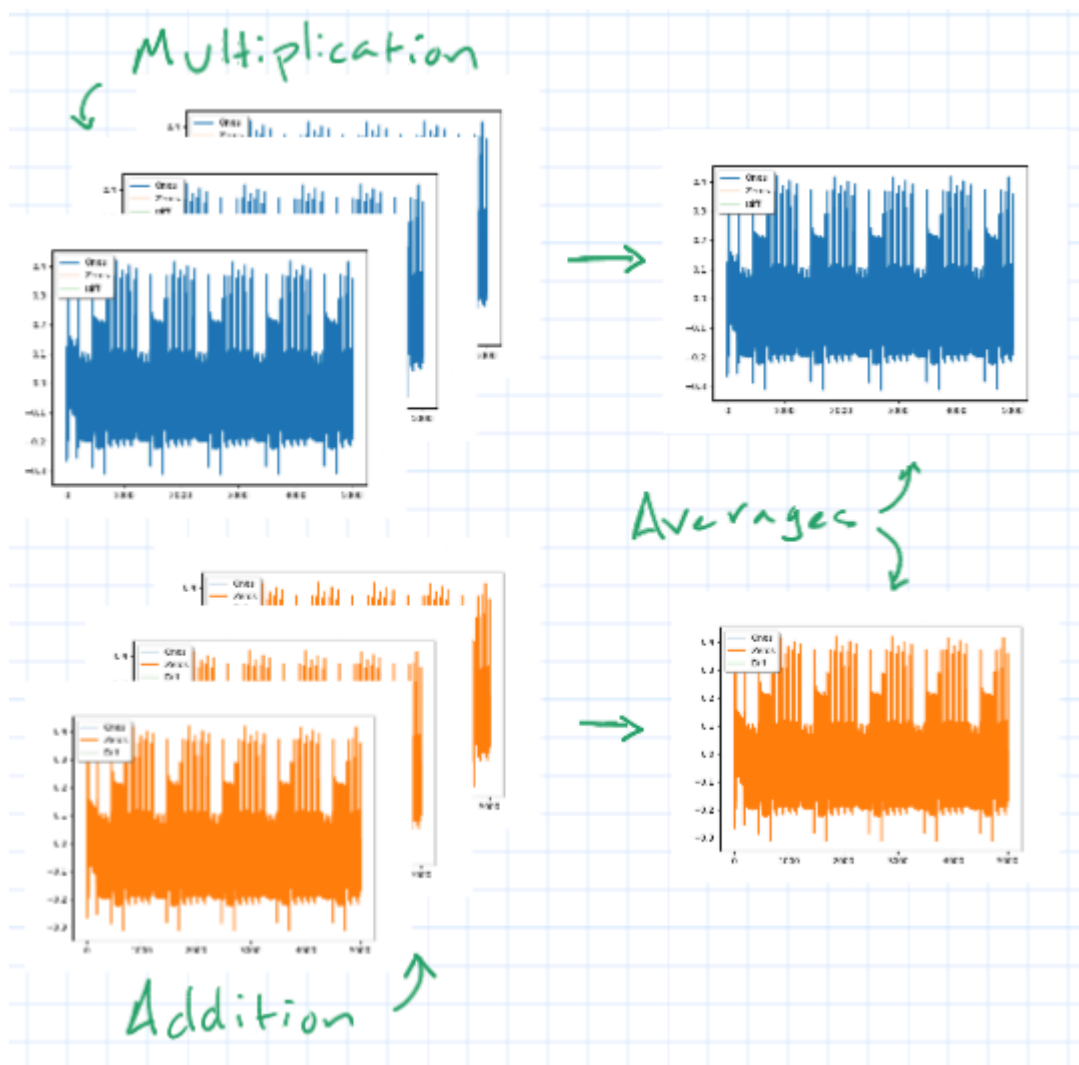


Let's start with a hypothetical example: pretend that we were trying to use power analysis to determine whether our microcontroller was conducting (1) addition or (2) multiplication at some point during a key operation. In fact, let's assume that we actually already *know* when it's adding and when it's multiplying, we're just trying to use the power traces to confirm this. At the start, our power traces are going to look nearly

indistinguishable; small amounts of noise will prevent us from saying, confidently, "This one shows multiplication", even if we know the exact point in time when that operation was occurring.

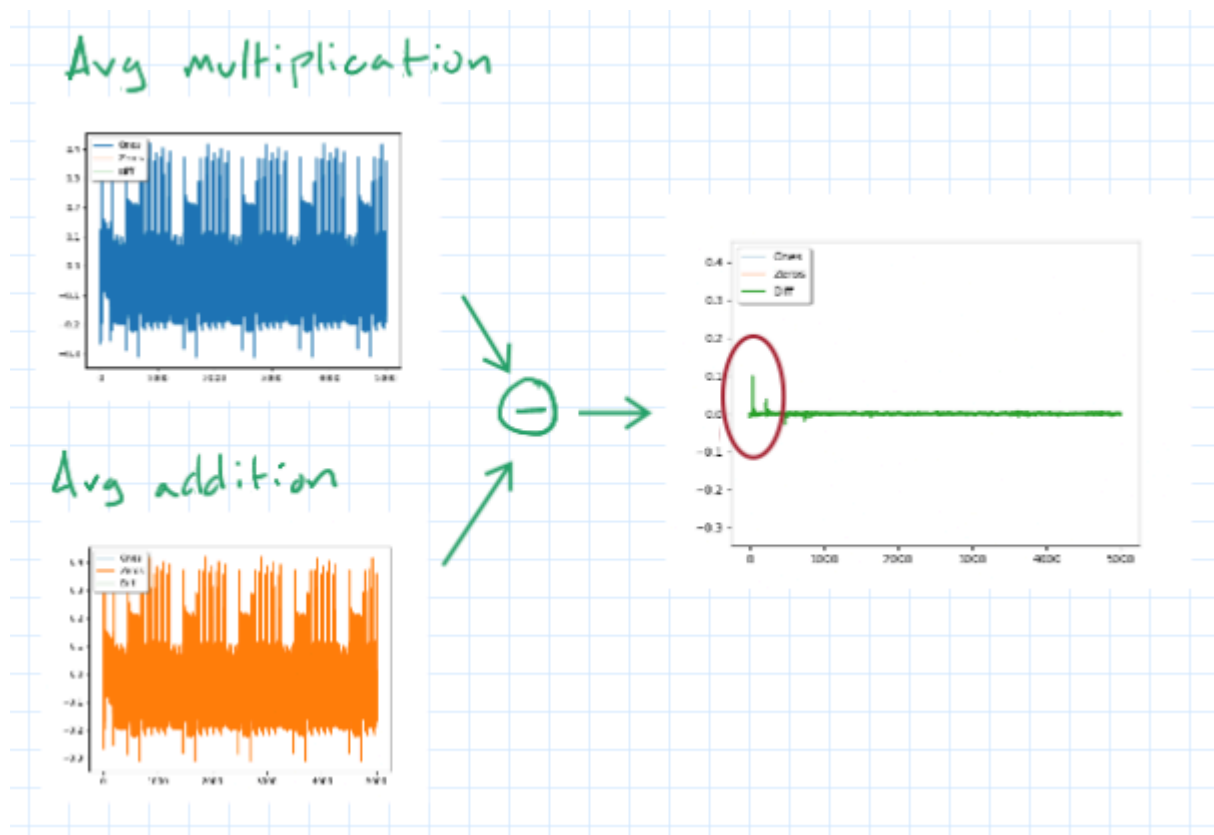


Since we know which traces correspond to addition and which to multiplication, though, we can group those sets of traces together. We can then cancel out much of the noise within our two sets by perform a point-by-point averaging of each group of traces.



As long as our microcontroller is deterministic (it is) and our traces all start at the same point in time relative to the operation in question (which we can engineer with the right trigger), the result will be two representative traces of our microcontroller as it was performing its key operation with the only (small) difference between them being how much power was being drawn while it was doing either addition or multiplication. If it takes more power for this microcontroller to perform multiplication than it does addition, the multiplication traces

will show a slightly higher power draw at exactly the point in time when that multiplication occurs, which we can identify by subtracting one average trace from the other. (NOTE: *This* is where the "differential" in DPA comes from.)



In fact (incredibly, perhaps), this same logic holds if what we want to do is simply differentiate between a set of traces where a *single bit in the microcontroller is a 1 or 0*. The resulting difference may be small but since the microcontroller's power consumption is directly and linearly correlated to how many bits are 1 we'll still be able to see it (though we may need a *lot* of traces for the noise level to become sufficiently small after averaging).

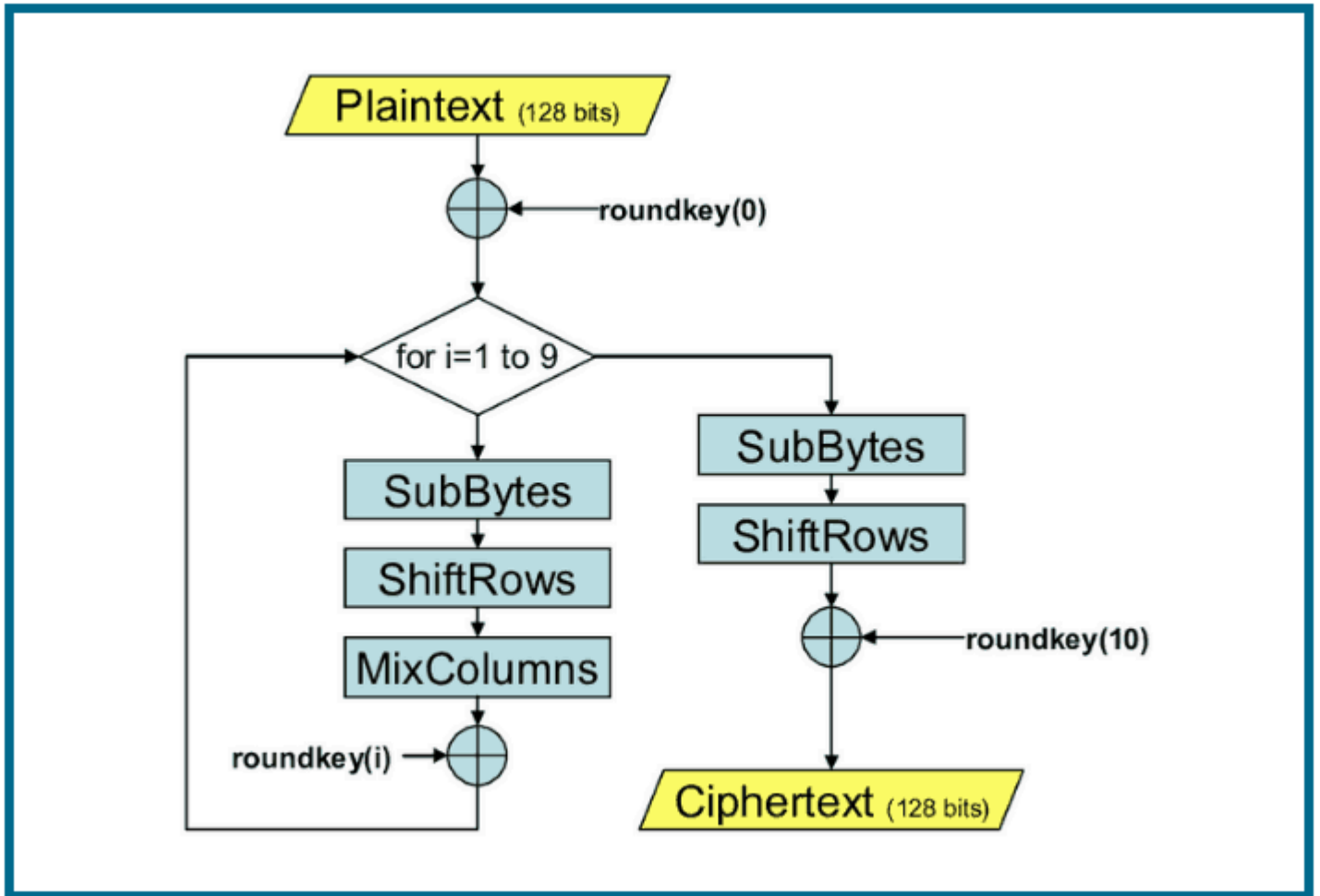
Let's briefly discuss how AES encryption works and then we'll get into the specifics of how to use this information to conduct our DPA attack.

A brief overview of AES encryption

AES ("Advanced Encryption Standard") is a highly-secure symmetric-key algorithm used to encrypt and decrypt sensitive data. According to [Wikipedia](#), it's the only publicly available cipher that's been approved by the US National Security Agency (NSA) for use in cryptographic modules handling top secret information. When everything works as its supposed to, AES provides **confidentiality** for a block of data, ensuring that only parties that know the private key are able to decipher the sensitive data. When used in an "authenticated encryption" mode, AES also provides **integrity** and **authenticity** by detecting when an encrypted message has been tampered with.

For AES128, a 16-byte/128-bit private key is used for encryption/decryption. To encrypt a chunk of data, the plaintext (in blocks of 16 bytes) is first XORed with the private key, one byte at a time (i.e. $pt[0] \wedge key[0]$, $pt[1] \wedge key[1]$, etc). Then, nine times in a row, the result of that XOR is passed to the functions SubBytes, ShiftRows, and MixColumns and the outputs are XORed with the next round key (determined via a [key](#)

schedule) . On the last round, the function MixColumns is omitted before the cipher text is generated.



The SubBytes function does nothing more complicated than convert a byte of input into a different byte of output via a look-up table, often referred as a "substitution box" or "sbox".

AES S-box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

The part we care most about is the output of SubBytes during the first round. Using the inputs pt (plaintext) and key (i.e. roundkey(0), the private AES key), we can express that output for each byte n (from 0 to 15) as follows.

$$\text{SubBytes}[n] = \text{sbox}[\text{pt}[n] \wedge \text{key}[n]]$$

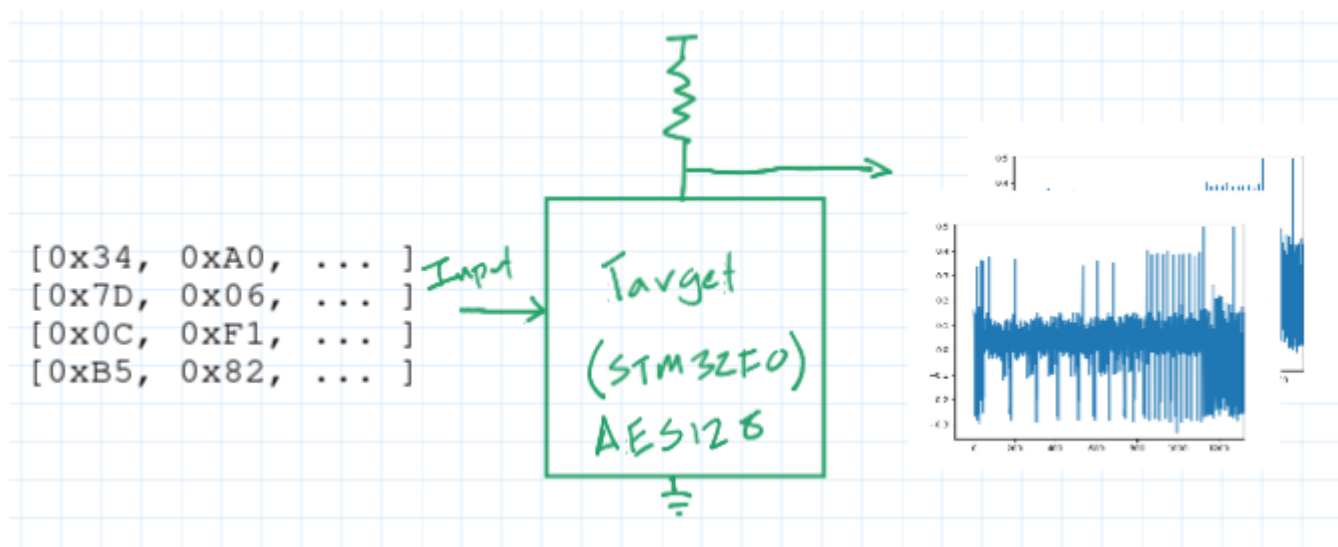
Conducting a DPA attack

So, where does that leave us? Regarding our device, we can control the plaintext that goes into the AES algorithm, but then we know nothing about the output. Argh, if only we knew the output of the first XOR block or the result of the first SubBytes function call! If we did, we would know enough to mathematically determine the key. For instance, if we knew that byte 0 of the input were 0x34 and that the output of that first SubBytes was 0x18, then

$$0x18 = \text{sbox}[0x34 \wedge \text{key}[0]]$$

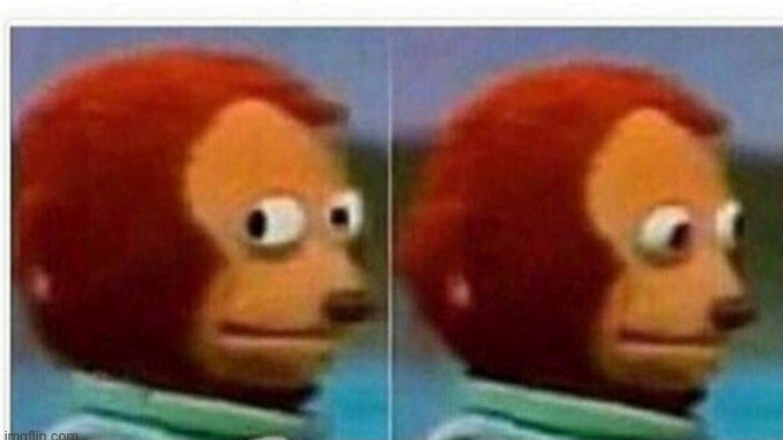
and we could easily show that $\text{key}[0] == 0x00$.

Wait, "if only we knew the value of a byte inside the microcontroller"?! That sounds a lot like the information we can get from a differential power analysis! So, to begin this attack, you'll record several thousand or so power traces of our electronic device doing AES128 encryption, making sure to vary the input and to store the input values alongside each power trace (that information is very important for later on).



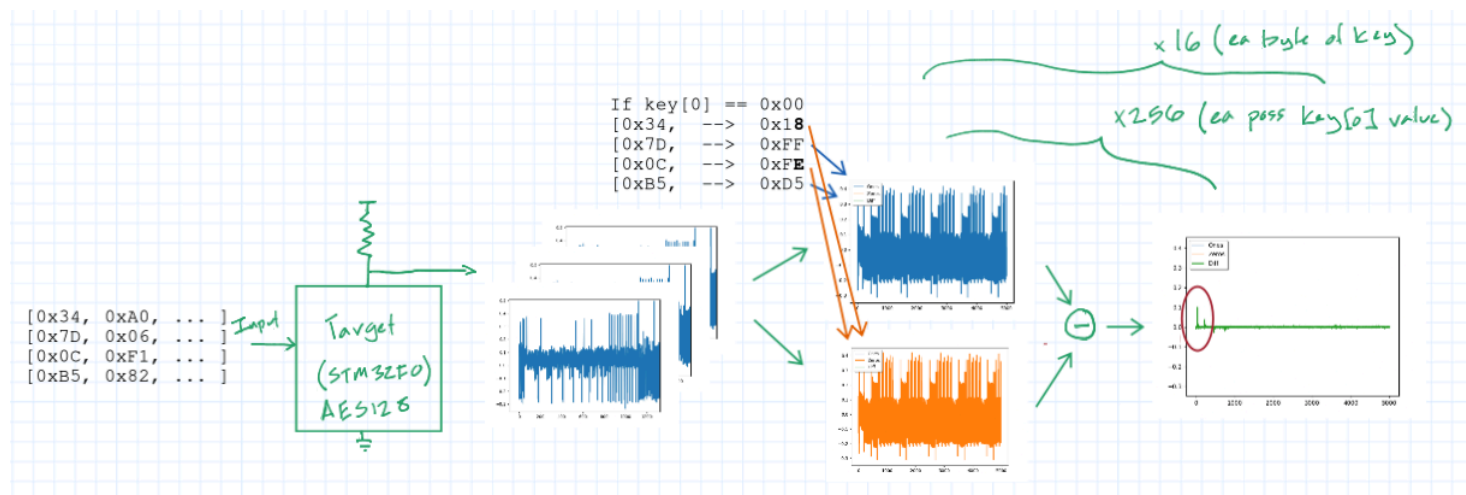
Alright, DPA, do your stuff! Tell us which bits are 1 or 0 so we can get the AES key!

DPA



Okay, so DPA doesn't *strictly* tell us the value of a byte inside the microcontroller, it just tells us if there's a difference in the power consumption between two sets of traces. We have to first sort our traces into two groups and then DPA will tell us if there's any difference between them. The last piece of the puzzle, then, is that we'll take a *guess* at the value of `key[n]`, sort the traces based on whether a certain bit in the output of `SubBytes` is a 1 or a 0 (which we can calculate, since we know the plaintext), and then use DPA to see if there's any actual difference between those two sets of traces.

In the graphic below, I've chosen to sort each trace by the value of bit 0, so `0x18` and `0xFE` each go into the group where `bit 0 == 0` and `0xFF` and `0xD5` go into the group where `bit 0 == 1`. (Interestingly, we don't even need to incorporate the values of `bits[7:1]` in our analysis, since just a single bit correlates strongly enough with the power consumption to be all that we need!)



If there isn't any difference between the average traces, it's most likely because we've guessed $\text{key}[n]$ incorrectly and accidentally sorted the two traces into our two groups completely randomly. But if there is a difference, then we can be confident that our key guess was correct!

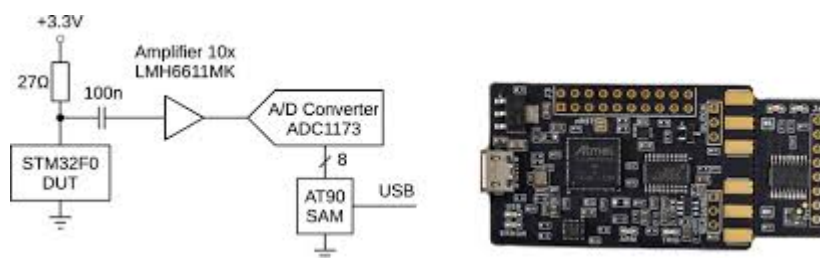
We'll have to potentially guess all 256 values for $\text{key}[0]$, and then do all that again 15 more times to get the rest of the AES key. But guessing 4,096 times in total is way better than trying to guess 2^{128} times!

Setting up the attack

If you're using a regular oscilloscope for this attack, you'll want to ensure that it can sample at 5-20x the clock speed of your target, with a bandwidth probably 5x times higher even than that. You can get away with sampling at the target clock speed if the samples are synchronized to that clock, however.

It's going to help greatly if you can script both your target and your oscilloscope, since we want to take several thousand power traces as quickly as we can. For a more thorough discussion of the subtleties of setting up this hardware, see [The Hardware Hacking Handbook](#).

Even easier is to use a ChipWhisperer from NewAE Tech, which has been designed for introductory hardware attacks such as this. In the next section, I'll demonstrate the DPA attack we've just discussed using my ChipWhisperer-Nano.



This neat little device contains a SAM4S microcontroller and external ADC to substitute for the oscilloscope and also a breakaway target (STM32) to run all of our attacks without needing a complex breadboard setup.

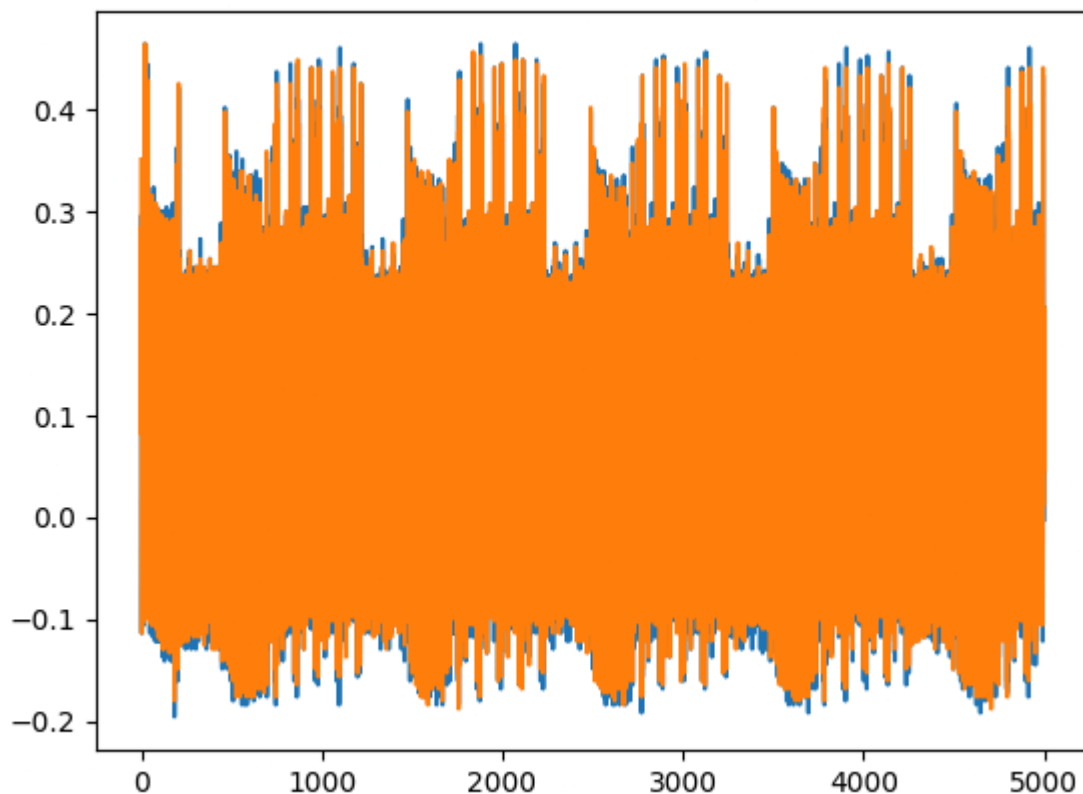
But the easiest of all is to use no hardware at all! The folks at NewAE Tech have a passion for teaching people about hardware attacks, so [their tutorials](#) include code to generate sample power traces that you can use to proceed with the attack in lieu of an actual target.

Conducting the attack

First, the STM32 on my ChipWhisperer-Nano is programmed to perform AES encryption. Note that the private key is set to the value below.

```
#define DEFAULT_KEY 0x2b,0x7e,0x15,0x16,0x28,0xae,0xd2,0xa6,0xab,0xf7,0x15,0x88,0x09,0xcf,0x4f,0x3c
```

Next, following the directions in [Lab 3-3 of SCA101](#), we take 2500 traces of the STM32 as it's performing AES encryption. What do a few of them look like?



Uhhh, hopefully DPA can do something with that??

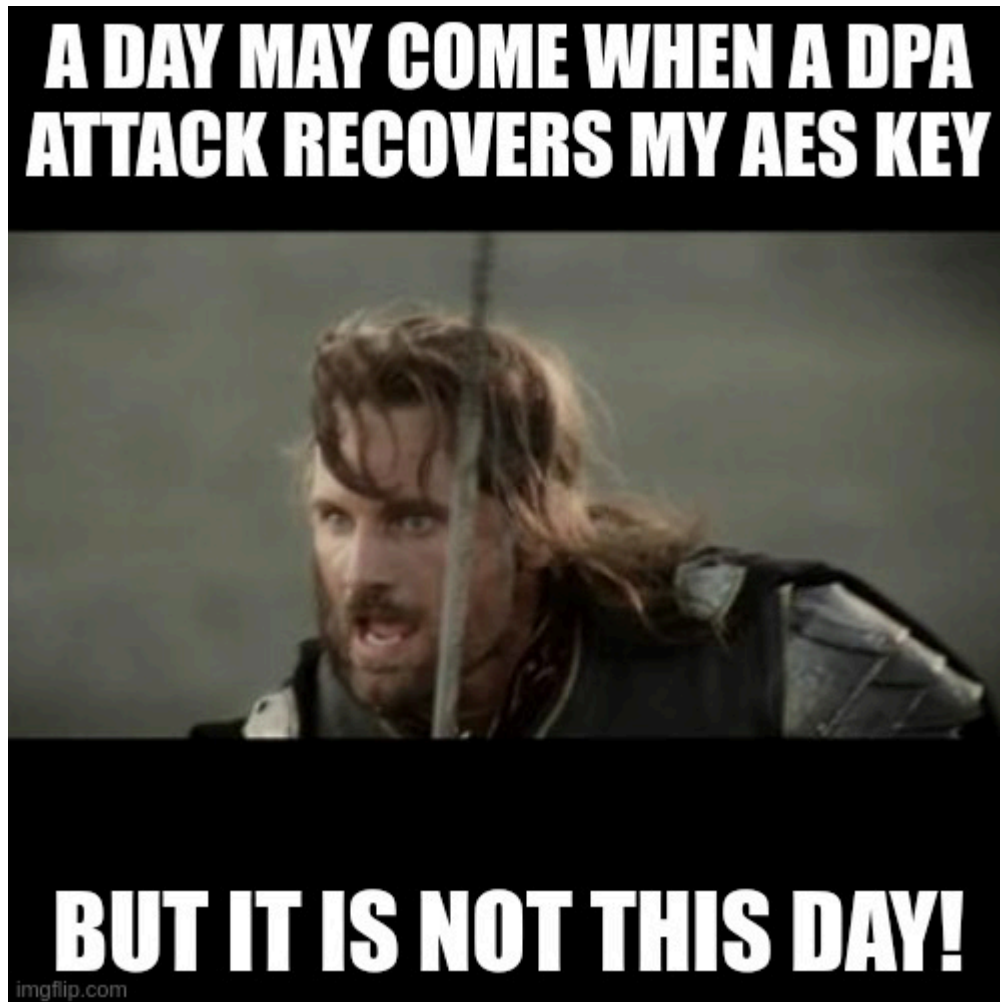
Once we have our traces, our DPA attack (in pseudocode) looks like this:

```
For each key byte, n [from 0-15]:
  For each possible key_guess [from 0-255]:
    For each trace:
      Calculate  $sbox[pt[n] \wedge key\_guess]$ 
      Sort the trace into one of two groups based on bit 0 of the result above
      Average the two groups of traces and then subtract them
      Find and store the maximum value of that trace
    Store the best guess for key[n] (which is the guess with the highest difference)
```

Here's a video of me conducting this attack with the ChipWhisperer-Nano. After collecting the traces (which took about a minute), it only takes **about six minutes** to recover the entire private key!! This is crazy, considering the "strength" of this encryption algorithm (and it would actually have gone *waaay* faster if my computer wasn't so stinkin' old).

Video link: <https://youtu.be/qaEAppkcRWc>

Defending against a DPA attack



I'd be remiss not to mention a few countermeasures for DPA attacks, though the short comments that follow only scratch the surface of how these attacks are mitigated in practice.

To begin, it might be that a DPA attack isn't actually something you need to worry about. Consider your threat model (and make one for your device if one doesn't exist!): what's the worst that would happen if an attacker recovered the AES key using DPA? If just one device becomes compromised (the one the attacker must have bought or stolen, since they have physical access to it) and all the attacker gets is open access to that device, then possibly no defense is needed.

If you conclude that defense *is* needed, though, then you have to do something that will undermine the attacker's assumptions about how they're able to conduct their attack.

For instance, we assumed in this article that the code performing the sbox lookup occurred at the *exact* same place in each of our power traces. If they hadn't, then averaging and subtracting traces wouldn't have shown us anything because the small peaks in our power traces that corresponded to the output of the sbox lookup would have shown up at different spots in each trace. This may actually happen accidentally if, for instance, an ISR were to fire between the attacker's trigger to start their oscilloscope and the actual AES encryption. We can make this happen intentionally, also, by **inserting random delays into our AES code**. Just double-check that your compiler doesn't optimize them out!

We also assumed in this article that the microcontroller's power consumption was correlated with the bits in the AES encryption. This is always true, strictly speaking, but the strength of the correlation decreases with additional noise in the system. If the microcontroller has **multiple hardware peripherals or multiple CPU cores running at the same time** as the encryption, this will add randomness to the power trace that will complicate a DPA attack.

Lastly, we could **use an AES implementation that is "side-channel resistant"**, which would employ a more complicated version of AES that leaks less information through a power trace.

Conclusion

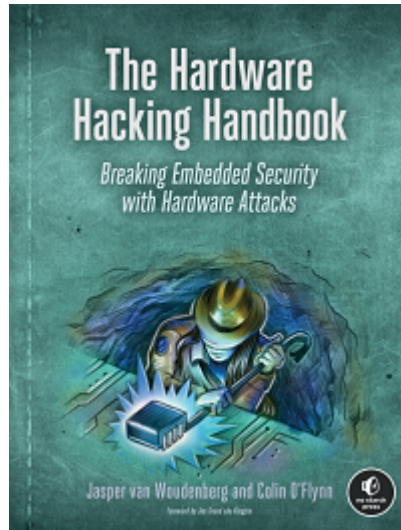
All electronic devices leak information about their internal operations via things like power consumption, timing, faulty outputs, etc; these are called "side channels". In some cases, this information is sufficient to compromise something important, like a private AES key. These are called "side-channel attacks."

One such attack on AES is based on a "differential power analysis", in which we use the slight difference between thousands of power traces of a device while its conducting AES encryption to determine the value of the private key. With the right tools, this process can take mere minutes.

Defending against a DPA attack requires updating one's threat model and, if needed, thwarting an attacker's ability to extract side-channel information from a power trace. The simplest defense is to insert a random delay during the AES operation to prevent the information being leaked (the output of the sbox in the first round, in this case) from occurring in the same temporal location each time a block of text is encrypted. More complex strategies include adding noise to the microcontroller's power consumption (by running other peripherals or CPU cores while the sensitive operation is occurring) or using a side-channel resistant version of AES.

Resources and going further

Much of the material in this article was developed by reading [The Hardware Hacking Handbook](#). I highly recommend it!



You can hear the authors of that book on the [Embedded.fm](https://www.embedded.fm) podcast in the following episodes:

- [Ep. 286: "Twenty Cans of Gas" \(Colin O'Flynn\)](#)
- [Ep. 431: "Becoming More of a Smurf" \(Jasper van Woudenberg\)](#)